Interview Preparation Document

Java Backend SWE1 – New Relic

Rohit Jangir

Contents

1 Coding Questions and Detailed Solutions **2**

2 System Design Short Notes (Backend + New Relic Style) **5**

3 Java Backend Theory Cheat Sheet **6**

4 Behavioral STAR Stories (Short) **7**

1 Coding Questions and Detailed Solutions

1. Maximum Subarray (Kadane's Algorithm)

Question: Given an integer array, return the maximum sum of any contiguous subarray.

Example:

$$[-2, 1, -3, 4, -1, 2, 1, -5, 4] \rightarrow 6$$

```
public int maxSubArray(int[] nums) {
    int current = nums[0], best = nums[0];
    for (int i = 1; i < nums.length; i++) {
        current = Math.max(nums[i], current + nums[i]);
        best = Math.max(best, current);
    }
    return best;
}
```

Explanation: At each position choose between starting new or extending previous subarray. Time: $O(n)$, Space: $O(1)$.

2. Longest Substring Without Repeating Characters (Sliding Window)

```
public int lengthOfLongestSubstring(String s) {
    Map<Character, Integer> map = new HashMap<>();
    int left = 0, max = 0;

    for (int right = 0; right < s.length(); right++) {
        char c = s.charAt(right);
        if (map.containsKey(c) && map.get(c) >= left)
            left = map.get(c) + 1;
        map.put(c, right);
        max = Math.max(max, right - left + 1);
    }
    return max;
}
```

Explanation: Maintain window without duplicates. Expand with right pointer, shrink with left.

3. Subarray Sum Equals K (Prefix Sum + HashMap)

```
public int subarraySum(int[] nums, int k) {
    Map<Integer, Integer> map = new HashMap<>();
    map.put(0, 1);
    int sum = 0, count = 0;

    for (int n : nums) {
        sum += n;
        if (map.containsKey(sum - k)) count += map.get(sum - k);
```

```
        map.put(sum, map.getOrDefault(sum, 0) + 1);
    }
    return count;
}
```

Explanation: Find count of previous prefix sums equal to $(sum - k)$.

4. Top K Frequent Elements (Heap)

```
public int[] topKFrequent(int[] nums, int k) {
    Map<Integer, Integer> freq = new HashMap<>();
    for (int n : nums) freq.put(n, freq.getOrDefault(n, 0) + 1);
    PriorityQueue<Integer> pq =
        new PriorityQueue<>((a, b) -> freq.get(a) - freq.get(b));

    for (int key : freq.keySet()) {
        pq.add(key);
        if (pq.size() > k) pq.poll();
    }
    int[] res = new int[k];
    for (int i = k - 1; i >= 0; i--) res[i] = pq.poll();
    return res;
}
```

5. Climbing Stairs (DP)

```
public int climbStairs(int n) {
    if (n <= 2) return n;
    int a = 1, b = 2;
    for (int i = 3; i <= n; i++) {
        int c = a + b;
        a = b;
        b = c;
    }
    return b;
}
```

6. Coin Change (DP)

```
public int coinChange(int[] coins, int amount) {
    int[] dp = new int[amount + 1];
    Arrays.fill(dp, amount + 1);
    dp[0] = 0;

    for (int coin : coins)
        for (int i = coin; i <= amount; i++)
            dp[i] = Math.min(dp[i], dp[i-coin] + 1);

    return dp[amount] > amount ? -1 : dp[amount];
```

```
}
```

7. Invert Binary Tree

```
public TreeNode invertTree(TreeNode root) {
    if (root == null) return null;
    TreeNode temp = root.left;
    root.left = root.right;
    root.right = temp;
    invertTree(root.left);
    invertTree(root.right);
    return root;
}
```

8. Diameter of Binary Tree

```
int diameter = 0;
public int diameterOfBinaryTree(TreeNode root) {
    height(root);
    return diameter;
}
private int height(TreeNode node) {
    if (node == null) return 0;
    int l = height(node.left);
    int r = height(node.right);
    diameter = Math.max(diameter, l + r);
    return Math.max(l, r) + 1;
}
```

Other Possible Questions

- Merge Intervals
- Meeting Rooms / Minimum Meeting Rooms (Heap)
- Validate Binary Search Tree
- Kth Largest Element (Heap / Quickselect)
- LRU Cache (LinkedHashMap)

2 System Design Short Notes (Backend + New Relic Style)

Design a Rate Limiter

- Requirements: prevent API abuse; fair access; fast lookup
- Use Sliding Window Counter / Token Bucket
- Store per-user counters in Redis
- Expire keys using TTL, atomic increments
- Track metrics: requests/sec, rejection count, p95 latency

Design Metrics Collection System (similar to New Relic)

- Clients send logs/metrics via async batching
- Collector service receives and pushes into Kafka / RabbitMQ
- Worker processes normalize and store into Time-Series DB (ClickHouse / Cassandra)
- Expose APIs for dashboards and percentile queries (p95/p99)

Design a Data Migration Service (aligns with real experience)

- Batch processing, multi-threading
- Queue for retries and DLQ
- Idempotency handling
- Metrics: throughput, failure rate, lag

3 Java Backend Theory Cheat Sheet

Java Collections

- HashMap: hashing, load factor 0.75, treeify after 8 collisions
- ConcurrentHashMap: segmented locking, thread-safe
- ArrayList vs LinkedList: random access vs insertion performance

Concurrency

- volatile = visibility guarantee
- synchronized = mutual exclusion + visibility
- Future vs Callable vs Runnable

Spring Boot

- DI / IoC container
- @RestController, @Service, @Repository, @Component
- Filter vs Interceptor vs AOP

Databases

- Indexing, ACID, joins, transactions
- Redis caching, TTL, eviction

4 Behavioral STAR Stories (Short)

Performance Improvement Story

Situation: Slow API causing timeouts
Task: Improve throughput
Action: Profiling, DB indexing, async processing, caching
Result: 20% faster response, improved customer experience

Data Migration Project

Processed 50K+ users, 15K/hour via batching + RabbitMQ, reduced errors.

Why New Relic

Real-time observability, improving reliability of distributed systems, interest in performance engineering.